

Topic 1

Introduction to Programming, Python and Development Environments

From engineering problem → algorithm → executable Python program

Primary: VS Code • Online: Google Colab

PYTHON 3

```
# Your first Python
program
print("Hello, Engineer!")

voltage = 12
current = 2.5
power = voltage * current
print(f"Power = {power}
W")
```

WHAT YOU SHOULD BE ABLE TO DO

Learning Outcomes

- Explain programming, algorithms and computational problem solving.
- Configure and use Python 3, VS Code and Google Colab.
- Write and execute basic Python programs using variables, data types, operators and input/output.
- Translate simple engineering calculations and algorithms into Python programs.
- Identify syntax, runtime and logical errors and use basic debugging techniques.

End-of-topic challenge

Build a small engineering calculator that accepts values from the user, performs a correct calculation, labels the result with units, and handles obvious input mistakes.

What Is a Computer Program?

Program

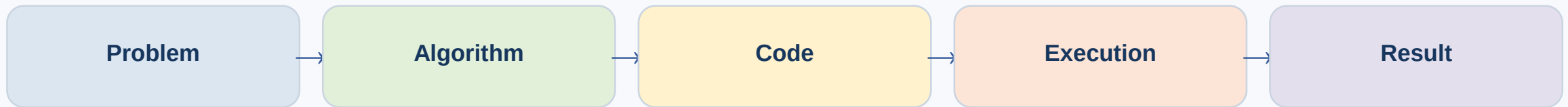
A sequence of instructions written in a programming language that tells a computer how to perform a task.

Programming

The process of analysing a problem, designing a solution, expressing it as code, testing it and improving it.

Programming Language

A formal language with rules (syntax) and meaning (semantics) used to communicate algorithms to a computer.



Engineering perspective: software is another tool for modelling, calculation, automation, control and data processing.

Computational Thinking for Engineers

- Decomposition — break a complex problem into smaller manageable parts.
- Pattern recognition — identify repeated structures or relationships.
- Abstraction — focus on information that matters; ignore irrelevant detail.
- Algorithm design — define a precise sequence of steps that transforms inputs into outputs.

Example: Electrical Power

Problem: Calculate DC electrical power.

Inputs: voltage V and current I .

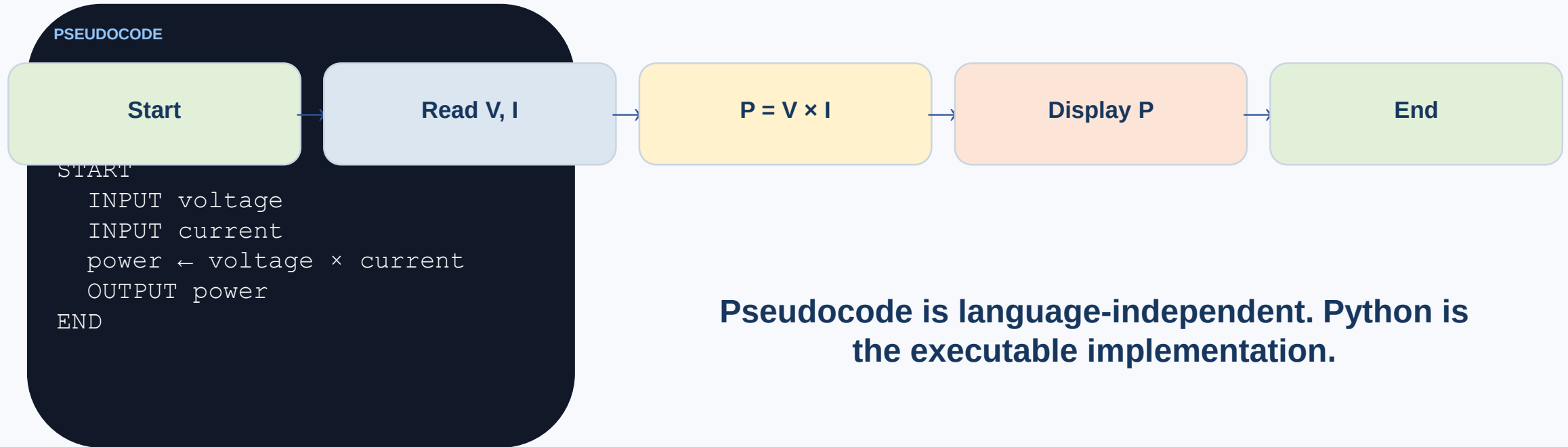
Process: $P = V \times I$.

Output: power P in watts.

Then ask: What if values come from a sensor? What if there are 1,000 readings? What if invalid data is entered?

Algorithms, Pseudocode and Flow of Execution

An algorithm should be finite, unambiguous, ordered and capable of producing the intended result.



Why Python?

- Readable syntax: students can focus on problem solving rather than excessive boilerplate.
- General-purpose: automation, web systems, data science, AI/ML, scientific computing, networking and scripting.
- Large ecosystem of libraries and a strong engineering/scientific community.
- Interactive: ideal for experimentation in a terminal or notebook.
- Object-oriented: supports the major OOP concepts that this course will develop later.

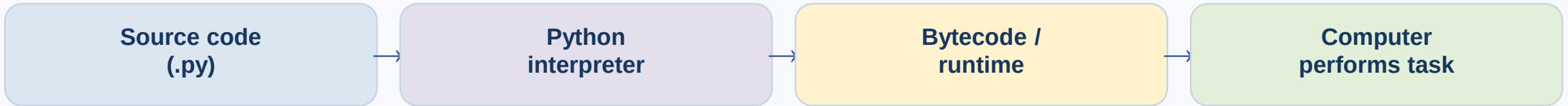
READABLE CODE

```
radius = 0.05          # metres
area = 3.14159 * radius**2
print(f"Area = {area:.6f} m²")
```

Important

Python is easy to start, but writing high-quality Python software still requires disciplined design, testing and practice.

How Python Executes Your Program



- You write source code in a text editor/IDE such as VS Code.
- The Python interpreter reads and executes the program.
- Errors may stop execution or produce an incorrect result.
- Python is dynamically typed: the type is associated with the value/object, not fixed permanently to a variable name.

Setting Up Python 3 and VS Code

- Install a current Python 3 release for your operating system.
- During installation on Windows, ensure Python is available on PATH (or use the Python launcher).
- Install Visual Studio Code and the Microsoft Python extension.
- Create a dedicated course folder; open that folder in VS Code.
- Select the correct Python interpreter from VS Code.
- Use the integrated terminal to verify Python and run scripts.

TERMINAL

```
python --version  
# or on some Windows setups  
py --version  
  
python hello.py
```

Recommended habit

One folder per exercise/project. Use meaningful filenames such as `power_calculator.py` rather than `test1.py`.

Your First VS Code Program

hello.py

```
# hello.py
print("Hello, UCU Engineer!")
print("Welcome to Python programming.")
```

- Create a file ending in .py.
- Save the file before running it.
- Run using the ► button or the integrated terminal.
- Observe output in the terminal.
- Modify the message, save, and run again.

Simple Question

Which part is Python syntax? Which part is a string value? What would happen if one quotation mark were removed?

Google Colab: Python in the Browser

Notebook structure

A Colab notebook contains code cells and text/Markdown cells. Code is executed cell-by-cell in a hosted runtime.

Best uses in this course

Quick demonstrations, experiments, short exercises, sharing examples and running Python when local installation is unavailable.

Do not rely on it for everything

Multi-file projects, local SQLite work and professional debugging are better learned in VS Code. Colab runtimes are temporary.

COLAB CODE CELL

```
x = 10
y = 4
print(x + y)
```

VS Code vs Google Colab

Feature	VS Code	Google Colab
Runs	On your computer	In a cloud runtime
Best for	Projects, files, debugging, SQLite	Demos, notebooks, quick experiments
Internet	Not required after setup	Normally required
Multi-file projects	Excellent	Possible but less natural
Persistence	Files stay locally	Runtime can reset
Course role	PRIMARY	COMPLEMENTARY

Python Syntax: Indentation Is Part of the Language

CORRECT

```
temperature = 85

if temperature > 80:
    print("Warning: high temperature")
    print("Check cooling system")

print("Monitoring continues")
```

INCORRECT

```
temperature = 85

if temperature > 80:
print("Warning")    # IndentationError
```

- A colon : usually introduces an indented block.
- Use consistent indentation—normally 4 spaces.
- Whitespace is not merely decoration in Python.

Comments, Identifiers and Naming

GOOD STYLE

```
# Calculate kinetic energy
mass_kg = 12.5
velocity_mps = 8.0
kinetic_energy = 0.5 * mass_kg *
velocity_mps**2
```

- Comments begin with # and should explain purpose or non-obvious reasoning.
- Identifiers name variables, functions, classes and other program elements.
- Names may contain letters, digits and underscores, but cannot begin with a digit.
- Python keywords such as if, for and class cannot be used as identifiers.
- Prefer descriptive snake_case variable/function names: sensor_voltage, total_cost.
- Python is case-sensitive: voltage and Voltage are different names.

Variables and Assignment

PYTHON

```
voltage = 12.0
current = 2.5
power = voltage * current

print(power)          # 30.0

voltage = 24.0        # name now refers to a
new value
```

Variable

A name that refers to a value/object. Assignment uses `=`. Read it as “assign”, not mathematical equality.

Dynamic typing

A variable name can later refer to a value of another type. This flexibility is useful, but careless changes can cause bugs.

Constants by convention

Python does not enforce ordinary constants. Use UPPER_CASE names, e.g., `GRAVITY = 9.81`.

Core Python Data Types

Type	Example	Typical use
int	42	counts, IDs, whole numbers
float	3.14159	measurements, engineering calculations
bool	True / False	conditions and states
str	"Sensor A"	text and labels
NoneType	None	absence of a value

TYPE EXAMPLES

```
samples = 120
voltage = 11.8
alarm = False
sensor = "A1"
last_reading = None

print(type(voltage))
```

Numbers: int and float

- int represents whole numbers of arbitrary practical size.
- float represents real-number approximations using floating-point arithmetic.
- Engineering measurements are commonly represented as floats.
- Floating-point values may contain tiny representation errors; do not assume every decimal is stored exactly.

NUMERIC TYPES

```
resistors = 6           # int
voltage = 5.0           # float
current = 0.125
power = voltage * current

print(power)
print(0.1 + 0.2)        # floating-point
reality
```

Engineering habit

Always keep track of units in names, comments or output: length_m, time_s, pressure_kpa.

Strings, Booleans and None

EXAMPLE

```
device_name = "Pump 3"  
status = "RUNNING"  
is_overheated = False  
last_error = None  
  
print(device_name)  
print(is_overheated)
```

- str stores text; use single or double quotes consistently.
- bool has two values: True and False. It is central to decision making.
- None represents “no value” or “not available yet”; it is not the same as 0 or an empty string.
- Use `type(value)` during learning/debugging to inspect a value’s type.

Operators and Expressions

Category	Examples	Purpose
Arithmetic	+ - * / // % **	calculation
Comparison	== != > < >= <=	compare values
Logical	and or not	combine/negate conditions
Assignment	= += -= *=	store/update values
Membership	in not in	test membership
Identity	is is not	test object identity

EXAMPLE

```
r = 10
area = 3.14159 * r**2
valid = r > 0 and area > 0

print(area, valid)
```

Arithmetic Operators: Engineering Examples

CALCULATIONS

```
# Ohm's law
voltage = 12
resistance = 6
current = voltage / resistance

# Circle area
radius = 0.25
area = 3.14159 * radius**2

# Quotient and remainder
full_boxes = 53 // 10
remaining = 53 % 10
```

- / always performs true division and returns a float.
- // performs floor division.
- % returns the remainder.
- ** performs exponentiation.
- Parentheses make intended order explicit and improve readability.

Check dimensions

A numerically correct formula can still be physically wrong if units are inconsistent.

Comparison Operators: Examples

CALCULATIONS

```
# Demonstrating Comparison Operators in Python
age1 = 20
age2 = 18
print("Age 1 =", age1)
print("Age 2 =", age2)
print("age1 == age2 :", age1 == age2)    # Equal to
print("age1 != age2 :", age1 != age2)    # Not equal to
print("age1 > age2  :", age1 > age2)      # Greater than
print("age1 < age2  :", age1 < age2)      # Less than
print("age1 >= age2 :", age1 >= age2)     # Greater than or
equal to
print("age1 <= age2 :", age1 <= age2)     # Less than or equal
to
```

comparison operators produce a Boolean result: True or False.

PRACTICE Logical Operators: Examples

CALCULATIONS

```
# Demonstrating Logical Operators in Python
age = 20
score = 75
# AND: Both conditions must be True
print(age >= 18 and score >= 60)
# OR: At least one condition must be True
print(age >= 18 or score >= 80)
# NOT: Reverses the Boolean result
print(not age >= 18)
```

and = ALL conditions must be true; or = AT LEAST ONE must be true; not = REVERSES the condition.

Member Operators: Examples

CALCULATIONS

```
# # Demonstrating Membership Operators
courses = ["OOP", "Networks", "Database", "AI"]
# Using 'in'
print("OOP" in courses)
# Using 'not in'
print("Python" not in courses)
```

Output

True

True

This is because "OOP" **is in** the list,
while "Python" **is not in** the list.

Membership operators are used to check whether a particular value **exists or does not exist inside a collection**, such as a list, tuple, set, dictionary, or string.

Member Operators: Examples

CALCULATIONS

Using membership with `if`

```
courses = ["OOP", "Networks", "Database", "AI"]
course = input("Enter course name: ")
if course in courses:
    print("Course is available")
else:
    print("Course is not available")
```

Output :
Course is available

Membership operators are used to check whether a particular value **exists or does not exist inside a collection**, such as a list, tuple, set, dictionary, or string.

PRACTICE Identity Operators: Examples

CALCULATIONS

Demonstrating Identity Operators

```
a = [10, 20, 30]
```

```
b = a
```

```
c = [10, 20, 30]
```

```
#
```

```
print(a is b)
```

```
print(a is c)
```

```
print(a is not b)
```

```
print(a is not c)
```

Output :True

False

False

True

Why? **b = a** means that **a** and **b** refer to the same list object. However, **c** is a different list object, even though it contains exactly the same value

Identity Operators: **is** and **is not**- check whether two variables refer to the **same object in memory**, not merely whether their values are equal.

Operator Precedence

EXAMPLES

```
result = 2 + 3 * 4      # 14
result2 = (2 + 3) * 4    # 20

# Kinetic energy
ke = 0.5 * mass * velocity**2
```

- Parentheses first.
- Exponentiation `**` before multiplication/division.
- Multiplication, division, floor division and modulo before addition/subtraction.
- Comparisons occur after arithmetic expressions.
- Logical not, then and, then or.
- When in doubt, use parentheses to communicate intent.

Input, Output and Type Conversion

POWER CALCULATOR

```
voltage = float(input("Enter voltage (V): "))
current = float(input("Enter current (A): "))
power = voltage * current
print(f"Power = {power:.2f} W")
```

- `input()` always returns text (`str`).
- Convert input when a number is required: `int(...)` or `float(...)`.
- `print()` displays output.
- f-strings embed expressions inside `{ }` and support formatting.
- `:.2f` displays a floating-point value to two decimal places.

Common beginner error

"12" + "3" produces "123" because both values are strings. Convert them before numerical addition.

Type Conversion: Explicit Is Better

CONVERSION

```
count_text = "25"  
count = int(count_text)  
  
temp_text = "36.5"  
temperature = float(temp_text)  
  
label = str(120)  
  
print(count + 5)  
print(temperature)
```

- `int("25")` → 25
- `float("36.5")` → 36.5
- `str(120)` → "120"
- `bool(...)` converts according to truth-value rules, but beginners should use it carefully.
- Invalid conversions raise errors, e.g. `int("twelve")`.

Errors: Syntax, Runtime and Logic

Syntax Error

Python cannot understand the program structure.

Example:
`print("Hello"`

Often detected before meaningful execution.

Runtime Error

The program starts but fails while executing.

Example:
`float("abc")`
`10 / 0`

Read the traceback from the bottom upward.

Logic Error

The program runs but gives the wrong answer.

Example: using $P = V + I$ instead of $P = V \times I$.

Testing and engineering reasoning are essential.

A program that runs is not necessarily a correct program.

Reading a Python Traceback

ERROR EXAMPLE

```
voltage = "12V"  
voltage = float(voltage)  
  
# ValueError: could not convert string to float:  
# '12V'
```

- Locate the file and line number mentioned.
- Read the exception type: ValueError, NameError, TypeError, etc.
- Read the final message carefully—it often explains the immediate problem.
- Inspect the values used on that line.
- Fix one problem, run again, and observe the new evidence.

Do not debug by random editing

Good Beginner Programming Habits

- Use meaningful variable names and include units where useful.
- Keep one statement/idea clear at a time; avoid clever but unreadable expressions.
- Use comments to explain intent, not obvious syntax.
- Save and run frequently while developing.
- Test with several values, including unusual or boundary values.
- Check whether the answer is physically/logically reasonable.
- Read error messages before asking someone else to fix the code.
- Type the code yourself—do not learn programming by only watching demonstrations.

Correctness → Readability → Testability → Maintainability

Guided Practical 1: Engineering Unit Converter

Task

Write a program that asks for a temperature in degrees Celsius and converts it to Fahrenheit and Kelvin.

$$F = (C \times 9/5) + 32$$

$$K = C + 273.15$$

Display all results clearly with units.

Skills practised

- `input()`
- `float()`
- variables
- arithmetic
- precedence
- f-strings
- formatting
- engineering units

Extension

Add conversion from Fahrenheit back to Celsius. Then test with 0°C, 100°C and -40°C. Explain why those are useful test values.

Guided Practical 2: Electrical Power Calculator

STARTER

```
# Complete and improve this program
voltage = float(input("Voltage (V): "))
current = float(input("Current (A): "))

power = voltage * current

print(f"Power = {power:.2f} W")
```

- Run the program with $V=12$, $I=2.5$.
- Predict the answer before running.
- Add resistance calculation $R = V / I$.
- What happens when $I = 0$? Why?
- Add energy calculation when operating time in hours is supplied.
- Use clear units in every output.

Discussion

Which failures are programming errors, and which are invalid engineering inputs?

Independent Practical: Sensor Reading Processor

- Ask the user for a sensor name, measured value and unit.
- Store each item in an appropriately named variable.
- Calculate a calibrated value using: $\text{calibrated} = \text{raw_value} \times \text{scale_factor} + \text{offset}$.
- Display a professional one-line result using an f-string.
- Use `type()` during development to inspect at least two values.
- Intentionally create one syntax error and one conversion error, observe the messages, then fix them.
- Run the program in VS Code; repeat the core calculation in Google Colab and compare the experience.

Submission evidence

1. .py source file
2. Screenshot/output from VS Code
3. Colab notebook or screenshot
4. Three test cases
5. Short note: which environment would you use for a larger project, and why?

Where to submit: Moodle

BEFORE WE MOVE ON Knowledge Check

- 1. What is the difference between an algorithm and a program?
- 2. Why does `input()` often need `int()` or `float()`?
- 3. What is the difference between `=` and `==`?
- 4. Why can indentation change a Python program?
- 5. Name the three broad categories of programming errors.
- 6. When is Google Colab useful, and when is VS Code preferable?
- 7. What value and type result from `7 / 2`? What about `7 // 2`?
- 8. Why should engineers label numerical outputs with units?